LectureLens — Multilingual AI Lecture Notes & Quiz Generator

CS3235 · Project option · 5 November 2026

Problem and scope

LectureLens helps students turn recorded lectures into structured study material: a timestamped transcript, hierarchical notes, and a five-question multiple-choice quiz — then translate that material on demand across six Indian/English languages. It uses AI4Bharat IndicConformer for speech recognition, BharatGen Param-1 for note/quiz generation, and AI4Bharat IndicTrans2 for translation. All three run as pluggable providers; the default demo configuration uses deterministic mock providers so the full pipeline can be exercised with zero model downloads and no external model API or key.

Example: upload a Kannada lecture → transcript in Kannada → notes and quiz generated in English (canonical generation language) → switch the workspace language to Tamil and receive cached Tamil notes/quiz without re-running the pipeline.

Supported languages: English, Hindi, Kannada, Tamil, Telugu, Malayalam. Out of scope for this MVP: authentication/multi-tenancy, word-level forced alignment, TTS, and live streaming ASR.

Models, data and suitability

Component	Selected resource	Purpose
ASR	AI4Bharat IndicConformer (.nemo via AI4Bharat NeMo, MIT)	Multilingual lecture transcription with chunk-level timestamps
Generative writing	BharatGen Param-1-2.9B-Instruct (Apache-2.0)	Hierarchical notes + 5 MCQs; bilingual hi/en only
Translation	AI4Bharat IndicTrans2 HF checkpoints (direction-specific)	Input translation to English; on-demand output translation
Voice activity detection	In-house energy VAD + FFmpeg silencedetect	Speech-aware chunking without a heavyweight VAD model
Official evaluation set	evaluation/test_manifest.csv, 59 cases / 6 languages	Supplemental WER/CER reference harness
Task-specific data	End-to-end API/browser runs; 42 automated tests	Evaluate this app's behavior

Param-1's model card lists language tags hi and en only. LectureLens therefore never feeds Kannada/Tamil/Telugu/Malayalam text to the LLM: the pipeline translates non-English sources to English first, generates notes and the quiz in English, then translates the generated material into the user's requested language. Structural identifiers (question ids, option letters A–D, correct_option) are preserved across translation so scoring stays correct.

IndicConformer's hybrid RNNT path exposes chunk/utterance timings, not word alignments; LectureLens stores segment start/end times at chunk level and does not claim word alignment. Long audio is split by the energy VAD with a small overlap, then de-duplicated after decoding.

Pinned model-card links and license notes are recorded in docs/model-selection.md. IndicConformer is MIT; Param-1 is Apache-2.0; IndicTrans2 checkpoints are distributed by AI4Bharat on Hugging Face.

Weights are never downloaded at application startup — real providers lazy-load and raise a clear ModelLoadError with configuration guidance if checkpoints are missing. This academic prototype links official sources and does not redistribute weights.

Application and prompting

```
Browser (React SPA)
   → FastAPI validation + storage
      → FFmpeg normalize / energy-VAD chunking
         → ASR (IndicConformer or mock)
            → input translation to English (IndicTrans2 or mock, if needed)
               → hierarchical notes (chunked Param-1 calls)
               → quiz generation (strict Pydantic schema + retries)
                  → output translation cache (on demand)
                     → validation → PostgreSQL persistence
```

The frontend provides upload with source/target language pickers, a processing view driven by real stage progress (completed stages / total stages — no fake timers), and a workspace with Transcript / Notes / Quiz tabs, language switching, quiz answering with scoring, and markdown/text export. Mock outputs are badged “demo mode” in the UI and reported by GET /api/v1/settings/providers.

Notes generation splits the transcript into bounded chunks, summarizes each chunk, then merges into hierarchical sections (heading, bullet content, source segment indices). Quiz generation asks for exactly five MCQs with four options each; a Pydantic validator rejects malformed JSON and retries. Generation prompts instruct the model to ground every statement in the transcript, preserve names/numbers/negation, and return only structured JSON. Long transcripts are handled by chunking rather than a single huge prompt; the local context budget is kept well below Param-1's max_position_embeddings of 8192.

Jobs run outside the HTTP request lifecycle: POST /api/v1/lectures stores the upload, creates Lecture + Job rows, enqueues an RQ task (or runs inline when INLINE_JOBS=true), and returns immediately. The frontend polls job status. Failed jobs always end as FAILED with a readable error_message. On-demand translation reuses a content_translations cache and never re-runs the full pipeline.

Measured evaluation

Evaluation	Cases completed	Micro WER	Micro CER	Other measure
Manifest transcripts (mixed-language sample)	59/59	9.21%	6.00%	Macro WER 9.20% / macro CER 5.37%
Backend automated tests	42/42	—	—	pytest pass rate 100%
Browser end-to-end (Playwright)	Full flow	—	—	Upload → notes → quiz submit → export → mobile viewport OK
Quiz structure (real API export)	1/1 quiz (5 Qs)	—	—	option_integrity 1.0; answer_skew 0.6

There were no service errors in these completed runs. The WER/CER figures above are computed by evaluation/evaluate_wer.py and evaluation/evaluate_cer.py against the references currently recorded in evaluation/test_manifest.csv. Because many sample hypotheses are lightly paraphrased rather than true

ASR outputs of human-verified transcripts, these numbers are harness sanity metrics, not published ASR benchmark scores. Real IndicConformer quality should be re-measured after loading official .nemo checkpoints and replacing references with independently reviewed transcripts.

WER is micro-averaged word edit distance over NFC-normalized whitespace tokens after punctuation stripping. CER uses character tokens with spaces removed (more appropriate for Indic scripts without reliable word segmentation). The evaluation suite also ships notes quality (keyword recall, section coverage, token Jaccard), quiz structure (option integrity, answer balance, distractor overlap), and translation metrics (corpus BLEU-4, chrF, TER) — all strictly reference-based; no metric fabricates scores.

The 59-case manifest covers English, Hindi, Kannada, Tamil, Telugu and Malayalam with greetings, instructions, technical content, numbers, dates and classroom announcements. Task-specific behavior is additionally covered by 42 unit/integration tests: upload validation, audio probing, segmentation, mock providers, hierarchical notes with a stub LLM, full API flow, failed-job → FAILED, translation caching, and quiz attempt scoring. Unit tests mock model responses; live E2E runs use the real mock-provider pipeline and real HTTP endpoints.

Latency in mock mode is dominated by FFmpeg + DB I/O (a 70 s demo upload completed the full multi-stage job in about one second). Real-model latency will be dominated by ASR and LLM decode and was not measured here because checkpoints are intentionally not downloaded in this environment.

Observed successes and failures

Case	Actual observation	Interpretation
70 s multi-burst demo WAV	3 transcript segments with chunk timestamps (0.0–22.5, 20.5–43.2, 41.2–70.0)	Energy VAD + overlap de-duplication split speech regions correctly
Hindi source, Kannada target	Stages included TRANSLATING_INPUT + TRANSLATING_OUTPUT; notes/quiz cached per language	Canonical-English generation + on-demand output translation works
Quiz attempt (always option A)	Score 0/5 against questions whose correct_option varied	Scoring uses stored correct_option, not client-supplied answers
Language switch to Tamil after job	POST …/translate reused cache; notes served from content_translations	Translation cache avoids pipeline re-run
Notes language badge	Initially showed “not translated yet” for kn/ta even when cached	Bug: available_note_languages only listed Note rows; fixed to union ContentTranslation languages
Mock translation tags	Headings show prefixes like [ta:ee02f9]	Expected in demo mode; UI badges mark mock providers so users are not misled
Segment timestamps	API returns start_time / end_time (not start/end)	Schema is explicit; chunk-level only, as documented
datetime.utcnow()	65 pytest warnings on first run	Replaced with timezone-aware datetime.now(timezone.utc); warnings down to 2
Quiz evaluation vs app schema	Generic evaluator expected key/is_correct	Fixed to accept app schema (id, correct_option) as well

Limitations are reported rather than hidden: default providers are mock (demo mode, not real ASR/LLM/NMT quality); Param-1 is hi/en only so non-English generation always round-trips through English; chunk-level timestamps only; single-user MVP with no auth; PDF export is client-side browser print; the 59-case evaluation references have not been independently reviewed by native speakers; Docker images ship the lightweight dependency set (real ML extras are documented but not installed by default).

Run and demo

1. ./setup.sh once (venv, Python deps, Postgres role/DBs, migrations, npm).
2. ./start.sh — API on :8000, frontend on :5173 (inline jobs; no worker needed). Open http://localhost:5173.
3. New lecture → upload an audio file → source language Hindi, target Kannada, translate enabled → Create.
4. Watch Processing (real stage progress) → open the workspace.
5. Inspect Transcript timestamps, switch Notes language (English ↔ Kannada ↔ Tamil), answer the quiz and submit for a score.
6. Export markdown; open Settings to confirm mock providers; run ./test.sh for the 42-test suite and ./status.sh / ./logs for operations.

Alternative full stack: cp .env.example .env && docker compose up --build (frontend on :8080, API on :8000). Evaluation commands are in evaluation/README.md. Fresh setup details are in README.md. Real-model provider switches and checkpoint paths are in .env.example and docs/model-selection.md. This report, architecture, implementation plan and model-selection notes live under docs/ and the repository root. The local Git repository can be initialized by the student; publishing requires their account. Add team identifiers in the course cover sheet/submission portal as required.

References

- IndicConformer ASR: https://github.com/AI4Bharat/IndicConformerASR
- AI4Bharat NeMo (loading path for .nemo checkpoints): https://github.com/AI4Bharat/NeMo
- BharatGen Param-1: https://huggingface.co/bharatgenai/Param-1-2.9B-Instruct
- IndicTrans2: https://github.com/AI4Bharat/IndicTrans2
- IndicTrans2 HF checkpoints: https://huggingface.co/ai4bharat
- FastAPI: https://fastapi.tiangolo.com/
- React + Vite: https://vite.dev/
- RQ (Redis Queue): https://python-rq.org/
- FFmpeg: https://ffmpeg.org/